
AST TPU 1.0 Documentation

Summer 2021

Tensor Processor with RISC and VLIW

—
Ashish Saroj Tondwalkar

Web: www.theashish.net

Abstract

The Tensor Processing Unit (TPU) is a category of new technology designed to compute neural networks by performing large amounts of matrix and vector processing. This paper provides the documentation of a TPU developed by Ashish Saroj Tondwalkar (AST) with the goal to enable engineers to write simple and easy assemblers/compiler. The scope of this paper is to describe the method of matrix processing and the details of the RISC processor is an outside discussion not covered here. The target audiences are peers and engineers with interest in TPU design who may find useful ideas.

Contents

1	Introduction	1
2	Architecture	1
2.1	Top Level Architecture	1
2.2	Matrix Processor Architecture	2
2.3	Systolic Array	3
3	Operations and Instruction Sets	4
3.1	Matrix Processor Instruction Set	4
3.2	RISC Instruction Set	5
4	Testing and Simulation	6
4.1	Testing Core Operations	6
4.2	Simulation Results	7

1 Introduction

A common strategy for developing novel computing technologies has been optimizing the hardware design for maximum compute speeds. However, this may lead to incompatibilities with software. The AST TPU 1.0 approaches the TPU design by prioritizing compatibility with software. The approach to software compatibility involves using several simple operations to divide the work of larger operations. Thus, the high level design of the TPU involves communication between a Matrix processor and a RISC co-processor.

2 Architecture

2.1 Top Level Architecture

The RISC processor and Matrix processor communicate with each other to complete their tasks (Fig. 1).

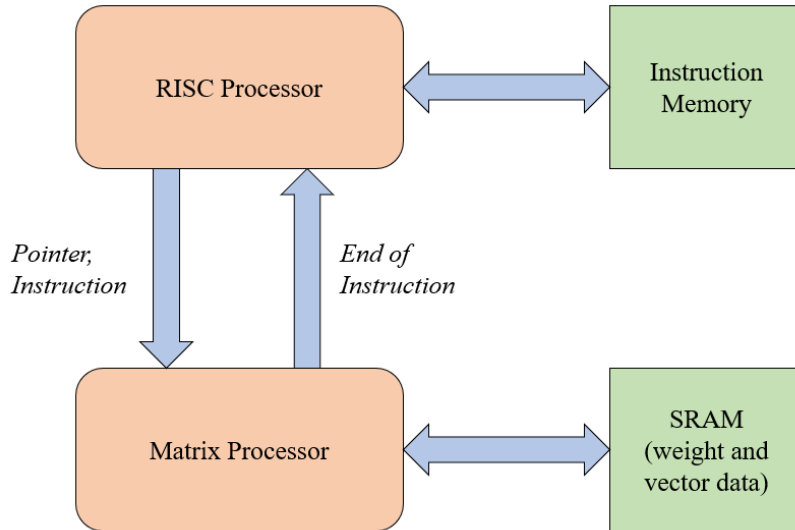


Figure 1: Top Level Architecture

The RISC processor is capable of basic operations necessary for logic, arithmetic, memory addressing, and peripheral interaction. Its main function is to provide a memory pointer, located in its register files, for the Matrix processor to read and write data from SRAM that contains weight data and input vector data. The RISC processor also relays the Matrix processor's instructions from the instruction memory. The Matrix processor communicates with the RISC processor by asserting that it has finished an instruction. All of the neural network computations are performed in the matrix processor.

2.2 Matrix Processor Architecture

The Matrix processor (Fig. 2) is divided into two sections: the data path and the control unit.

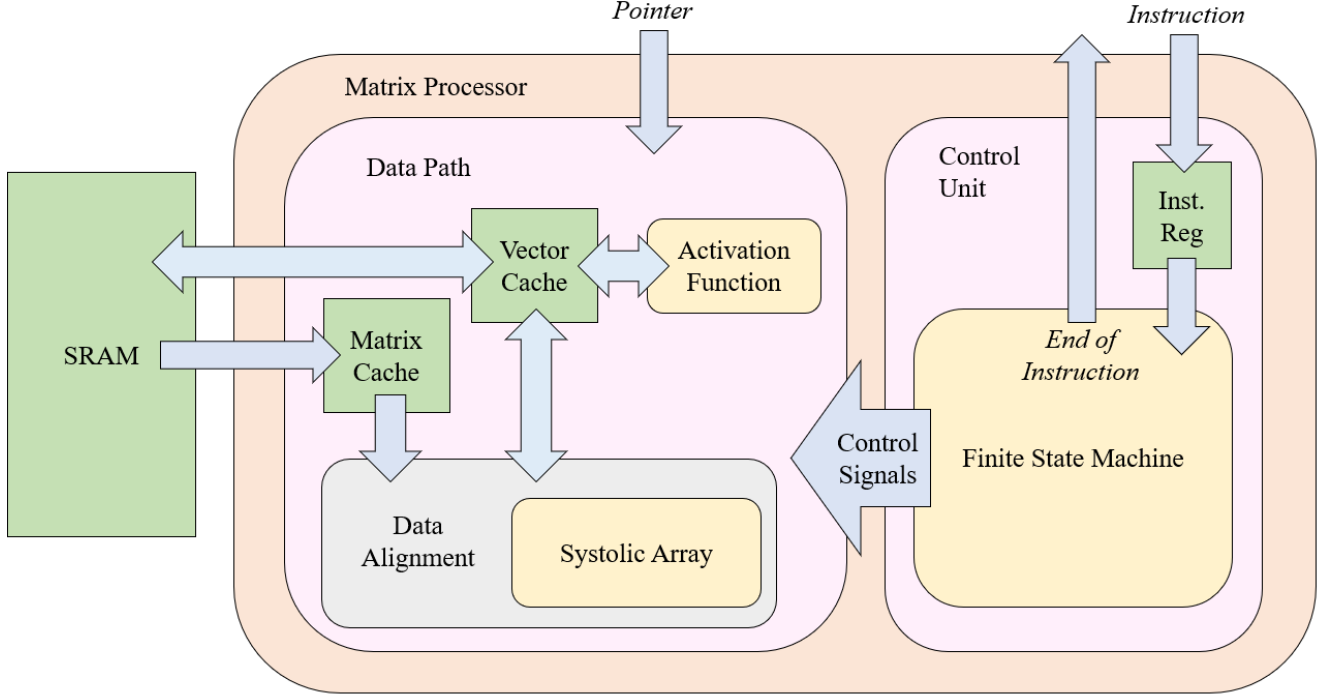


Figure 2: Matrix Processor Architecture

The Control Unit determines the necessary control signals to be asserted for each instruction. The control unit stores the Matrix processor's instruction word and processes the word through a finite state machine (FSM). Depending on the instruction relieved, the FSM will send various control signals to the data path at different machine cycles. When the instruction has complete all of its machine cycles, *End of Instruction* signal is asserted and sent to the RISC processor.

The Data Path handles all of the neural network data and performs the matrix operations. All data handled by the Data Path are 16 bit floating point words to reduce data loss from multiplication. Data is retrieved from the SRAM using a pointer received from the RISC processor and is stored into internal cache. The weight data for neural network layers are stored into the matrix cache. The input/vector data is stored into the vector cache. The data stored in cache is connected to a alignment module which prepares the data to be cascaded into the systolic array during matrix multiplication. The matrix multiplication takes place inside the systolic array, which contains multiply-accumulate units that return the resulting vector to the vector cache. Vectors may also be processed in the Activation Function module that performs a Rectified Linear Unit (ReLU) operation and returns the resulting vector to the vector cache.

2.3 Systolic Array

The main module that computes the multiplication is the systolic array (Figure 3). The systolic array consist of several 16 bit floating point multiply-accumulate (MAC) units that stores the final product. Every clock cycle, data is cascaded into the MAC units and the MAC units cascade their current product to the next MAC unit. Because the data needs to be cascaded in such a way, there is a data alignment module that properly connects the matrix cache to the systolic array.

Since the AST TPU performs a matrix-vector multiplication of n sized vector, the size of the systolic is also size n . Depending on the amount of resources available on the hardware, the floating point multiplication and addition requires q number of clock cycles. Consequently, the full multiplication requires a total number of $q(2n-1)$ clock cycles.

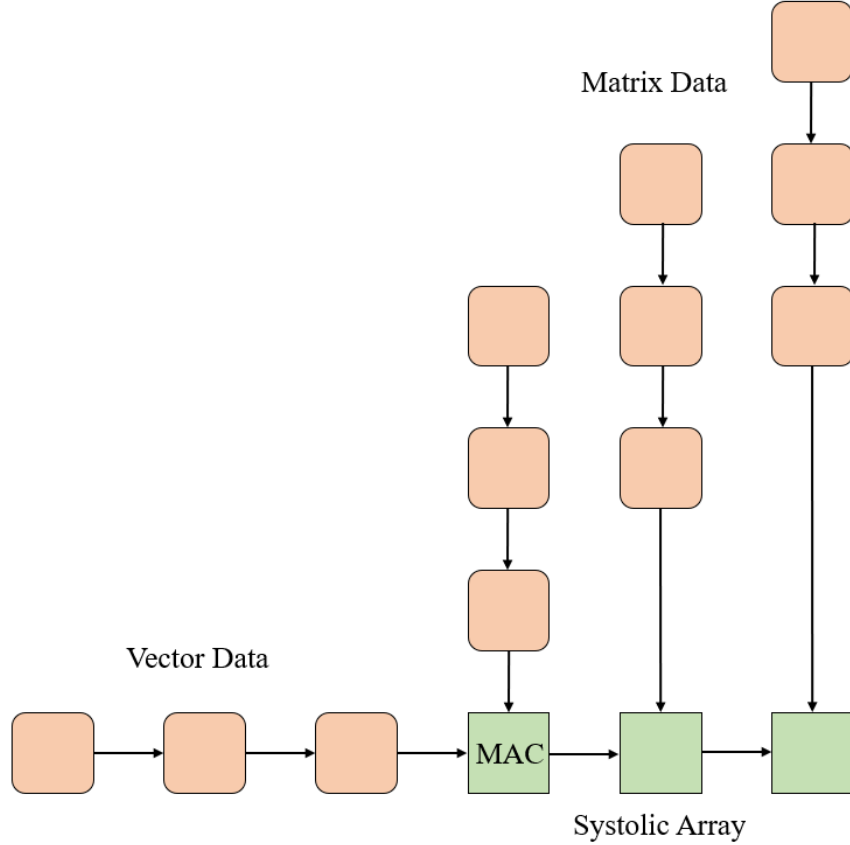


Figure 3: Systolic Array Methodology

3 Operations and Instruction Sets

The instruction word provided to the TPU is 16 bits long; the most significant 8 bits are the matrix processor's instruction word and the least significant 8 bits are the RISC processor's instruction word.

3.1 Matrix Processor Instruction Set

The Matrix processor has the necessary instructions to load data, compute the multiplication and activation function, and store the data (Table 1). The operations involving data load and storage use a pointer provided by the RISC processor.

Instruction	Description
NOOP	No Operation, Do nothing for an instruction cycle
LDV	Load data from SRAM into vector cache
LDM	Load data from SRAM into matrix cache
MXV	Perform matrix multiplication and store result in vector cache
STV	Store vector from cache into SRAM
RELU	Perform ReLU activation function from vector cache and store result into vector cache

Table 1: Matrix Processor Instructions

These operations can be used together compute a neural network layer. Since the vector result is stored into the vector cache, several layers can be computed by reloading weight data into the matrix cache and performing another multiplication (Table 2).

One Layer	LDV -> LDM -> MXV -> STV
One Layer with ReLU	LDV -> LDM -> MXV -> RELU -> STV
Multiple Layers	LDV -> LDM -> MXV -> LDM -> MXV -> ... -> LDM -> MXV -> STV
Multiple Layers with ReLU	LDV -> LDM -> MXV -> RELU -> LDM -> MXV -> RELU -> ... -> LDM -> MXV -> RELU -> STV

Table 2: Approaches to Neural Network Layer Computation

3.2 RISC Instruction Set

The RISC processor is comprised of mostly standard instructions (Table 3) necessary to function as a central processing unit. Note, the custom instruction *WAIT* that allows for the Matrix processor to complete its time consuming instructions before moving on to the next VLIW. This instruction is usually used for matrix processor instructions that cannot be completed before the RISC processor instructions are completed, such as LDM or MXV.

Instruction	Description
ADD	Add source and destination register
SUB	Subtract destination from source register
INC	Increase destination register by K
DEC	Decrease destination register by K
XOR	Bitwise XOR source and destination registers
AND	Bitwise AND source and destination registers
OR	Bitwise OR source and destination registers
CPY	Register to register transfer
SHRA	Shift right arithmetic by K
SHLL	Shift left logical by K
WAIT	Wait for matrix processor before fetching next instruction
LD	Load data from memory into destination register
ST	Store data from source register into memory
JUMP	If condition is satisfied, jump program counter to new address
IN	Store data from input peripheral into destination register
OUT	Output data from source register to output peripheral

Table 3: RISC Processor Instructions

Since the RISC processor is not meant to perform the same tasks as a central processing unit, the main purpose of the instructions is to manipulate the data pointer for the matrix processor. The data pointer is stored in a register file and can be manipulated using arithmetic, logic, or loading the pointer directly from memory (Table 4). Realistically, the best method is to load the pointers for the matrices because manipulating the pointer with multiple instructions can take an unnecessary number of clock cycles.

Arithmetic	INC by 2 -> ADD R0, R0 -> INC by 1
Logic	INC by 2 -> SHRA R0 -> CPY to R1 -> XOR R0, R1
Loading	LD R0

Table 4: Possible Methods for Manipulating Data Pointer

4 Testing and Simulation

4.1 Testing Core Operations

A test was performed for an algorithm to compute a neuron layer (Table 5). The TPU was simulated using a 4x4 matrix and a 4x1 input vector (Fig. 4).

Step	TPU	RISC
0	No Op	Wait
1	Load data into vector cache	Wait
2	No Op	Change pointer to 4
3	Load data into matrix cache	Wait
4	Multiply	Wait
5	No Op	Change pointer to 20
6	Store vector data into memory	Wait

Table 5: Steps of the Test Algorithm

$$\begin{bmatrix} 6 \\ 6 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & -1 & 3 & 5 \\ 1 & 3 & 0 & 4 \\ 3 & 0 & -1 & -2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ -1 \\ 1 \end{bmatrix}$$

Figure 4: Matrix Multiplication Test Equation

The algorithm begins by loading the first 4 words from memory into the vector cache. Second, the pointer is changed to the value 4. Starting from word location 4, the proceeding 16 words from memory are loaded into the matrix cache. The vector and matrix are then multiplied. Then the pointer is updated to value 20. Finally, the vector data is stored into memory starting at location 20.

4.2 Simulation Results

Simulations results (Fig. 5) reveal that the TPU was able to complete the algorithm without freezing or unexpected behavior.

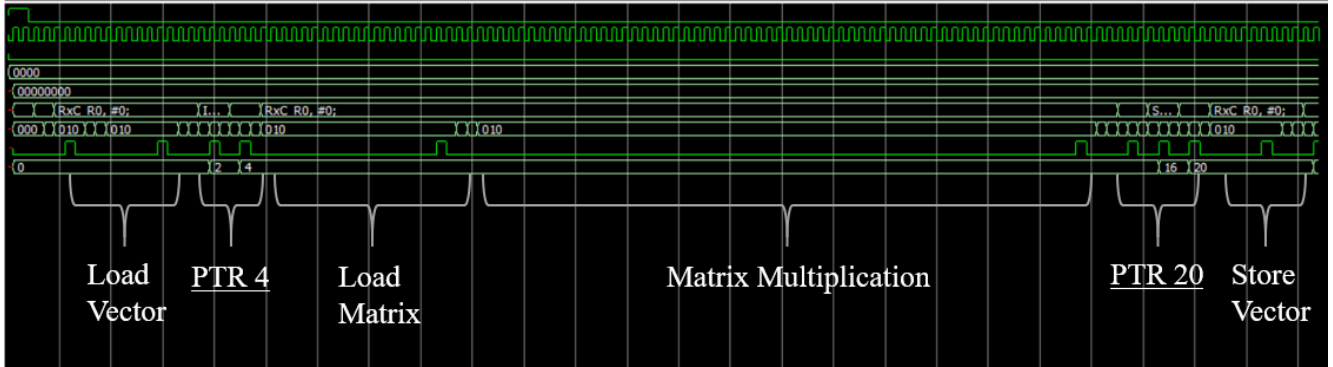


Figure 5: Simulation Waveform

Inspecting the contents of the SRAM (Fig. 6), it is observed that the first row is comprised of the input vector data and the next 16 words are the matrix data. This data was initialized in the SRAM prior to simulation. The last row beginning at location 20 is the data written by the TPU. Although the data is formatted in 16 bit floating point, converting it to decimal reveals that the output vector data matches the expected vector result from the matrix equation.

Address Location	0	0100000000000000	0000000000000000	1011110000000000	0011110000000000	Input Vector
	4	0100000000000000	1011110000000000	0100001000000000	0100010100000000	Matrix/Weight Data
	8	0011110000000000	0100001000000000	0000000000000000	0100010000000000	
	12	0100001000000000	0000000000000000	1011110000000000	1100000000000000	
	16	0000000000000000	0000000000000000	0000000000000000	0011110000000000	
	20	0100011000000000	0100011000000000	0100010100000000	0011110000000000	Output Vector
		6	6	5	1	

Figure 6: Memory Contents